Worksheet: Machine Learning in Production

CS 5090 · Software Engineering with Generative AI · Module 2 (Lectures 1–8) · collected for completion credit. Discussing with your neighbor is encouraged; write your own answer. Every question is close to a Key-takeaways bullet from one of the eight lectures.

Name: _____

Part 1 · The model is a small box; when to use ML; goals

recall **1.1** The transcription start-up: the speech-recognition model is about 5% of the work. Name **four** components of the system that are not machine learning.

match **1.2** Five things that make ML systems different from ordinary software. Write the property next to its consequence: **no specification** · **mistakes are certain** · **the world moves** · **data is code** · **entanglement (CACE)**.

Consequence	Property
A system can get worse with no deployment and no code change	
You cannot write a unit test that says "the output is correct"	
99% accuracy is a design input, not a victory	
The training set needs a version number, like a library	
Changing one feature changes every weight downstream	

decide **1.3** A bank wants a model that automatically closes accounts it suspects of fraud, with no appeal. Which of the five "when ML is the right tool" conditions fails, and what single design change would make ML acceptable here?

order **1.4** Number the four layers of goals from top (1) to bottom (4): ___ model quality (pass@1) ___ organisational goal (revenue) ___ user outcome (did they get what they came for?) ___ leading indicator (daily active use).
Booking.com's 23 models: what did the scatter plot of model improvement against conversion rate show?

Part 2 · Requirements and planning for mistakes

fill in **2.1** Fall detection on a smartwatch. Fill in the three parts: the **world** event, the **assumption** that connects them, and what the **machine** actually detects. Then name one way the assumption fails.

match **2.2** Four kinds of "requirements": **system** · **user** · **environment assumption** · **component spec**. Label each:

Statement	Kind
The model returns a confidence score with every transcript segment	
Users upload audio from a quiet room with one speaker	
A customer can find any sentence in their meeting within a day	
A journalist needs speaker names attached to quotes	

apply **2.3** Gmail suggests three short replies but never sends one. Which of the six "designing for failure" patterns is that? Name **one more** pattern and give an example for a coding agent that edits your repository.

work it out **2.4** A coding agent pushes a patch that deletes a database table in production. Fill in the risk analysis, working backwards from the harm:

Harm	Hazard	Cause	Control (non-ML)
production data lost			

Part 3 · Architecture and quality assurance

decide **3.1** Five quality attributes decide where a model lives: latency, cost, privacy, availability, updatability. A hospital's diagnostic model and a free photo-filter app: where does each model live (server, on device, hybrid), and which attribute decides it?

recall **3.2** True or false, circle one each:
T / F Training/serving skew usually shows up as an exception at serving time.

T / F The structural fix for skew is one shared feature-encoding component used by both training and serving.
T / F In an ML pipeline, the "evaluate" step should be a gate that can stop deployment on its own.

classify **3.3** Four levels of QA: **model** · **data** · **pipeline** · **system**. At which level does each defect live?

Defect	**Level**
90% overall accuracy, 40% on the rare category	
A join misaligns rows by one; nothing crashes; the model is worse	
Accuracy in production falls for 16 weeks; the model file never changed	
The guardrail that should block nonsense never actually fires	

recall **3.4** Behavioural tests for a model with no specification. Give one example each of an **invariance** test and a **directional** test for a sentiment model.
Why can a model only be *validated*, never *verified*?

Part 4 · Testing in production, versioning

match **4.1** Four safe ways to ship: **shadow** · **canary** · **A/B test** · **feature flag**.

Description	Method
Serve the new model to 1% of traffic, watch, expand or roll back	
Run the new model in parallel on real requests but serve the old answers	
Turn the new model off in seconds without a deployment	
Split traffic deliberately to measure a difference with statistics	

recall **4.2** Two rules of an A/B test that must be fixed *before* it starts: _____ and _____. What goes wrong if you stop the test the first time the result looks good?

---

fill in **4.3** A customer asks why they were refused a loan eight months ago. Fill in the provenance chain from right to left: prediction ← _____ ← _____ ← _____ ← source.
What is the five-minute fix that makes the first link possible?

---

think **4.4** A coding assistant: give one **natural label** (a signal the user produces anyway) that tells you a suggestion was wrong. Why is "acceptance rate" alone a trap?

Part 5 · Process, technical debt, responsible engineering

match **5.1** Five kinds of ML technical debt: **glue code** · **configuration debt** · **undeclared consumers** · **entanglement** · **data dependencies**.

Situation	Kind
You improve your model; a team you never met has an incident because they tuned around the old behaviour	
An upstream feed changes its date format and your accuracy drops	
Hundreds of untested, undocumented knobs	
Adding one feature shifts every weight in the model	

recall **5.2** "We retrained on newer data; it is a small model change, so we only re-ran the model's unit tests." What is wrong with this, in one sentence? (Hint: CACE.)

---

match **5.3** Three fairness definitions: **anti-classification** · **independence** · **separation**. Which one is "equal error rates across groups"? _____ Which one is "do not use the protected attribute"? _____ Why is the last one weak?

---

recall **5.4** In one sentence each: the difference between **reliability** and **safety**; and the four questions asked after an ML system goes wrong (disclosed? …).

---

decide **5.5** Generated code that a student merged causes an outage. Who owns it, according to the lecture, and why is that a professional claim rather than a legal one?
